

Training Binary-Weight Diffusion Models from Scratch: A Controlled MNIST and CIFAR-10 Study

Ayushman Bhattacharya
Pollinations AI
ayushman@pollinations.ai

Nihal Gazi
Dept. CSE AI & ML, IEM
nihalg2006@gmail.com

Abstract—At one bit per weight, post-training quantization (PTQ) can destroy a diffusion model’s denoising function. Does this imply that a useful binary score network must be distilled from a floating-point teacher? We isolate that question in a small residual U-Net whose parameterization is identical across precision conditions. We compare FP32 training, zero-shot PTQ, binary-weight training from random initialization, floating-point warm-started quantization-aware training (QAT), and a binary-weight/binary-activation core. We additionally cross mean-centering with pre- versus post-activation residual ordering. All conditions use the same data, optimizer, per-stage budget, and ancestral 1,000-step DDPM sampler and are repeated over three training seeds; warm QAT additionally inherits a 12-epoch FP32 checkpoint. Evaluation uses a held-out domain encoder and reports Fréchet and kernel distances, density, coverage, and class-prior divergence, together with real–real and Gaussian-noise controls. On MNIST, the best native W1A32 condition obtains domain-feature Fréchet distance 46.33 ± 17.24 , far below zero-shot PTQ (1314.20 ± 314.96), but well above FP32 (3.15 ± 0.38). On CIFAR-10, all conditions fail the control audit: even FP32 obtains 382.67 ± 46.91 versus a real–real baseline of 7.13. The previously proposed centering explanation is not supported: centering increases latent-to-binary angular error and worsens the pre-activation model. Thus, native optimization substantially mitigates PTQ collapse on MNIST, but neither establishes parity with floating point nor generalization to natural images. We release code, per-seed checkpoints, hashes, histories, controls, and physically bit-packed inference weights. Our conclusion is deliberately bounded: training path matters at one bit, while a general teacher-free solution to binary diffusion remains open.

Index Terms—binary neural networks, diffusion models, quantization-aware training, post-training quantization, reproducibility

I. INTRODUCTION

Diffusion models learn to reverse a gradual Gaussian corruption process and have become a standard approach to image generation [1]. Their repeated denoising passes make model memory and arithmetic especially relevant on limited hardware. Binary neural networks replace many floating-point weights with signs and per-channel scales [2], [3]. Such weights can be stored compactly and can, with specialized kernels, admit binary arithmetic. The latter qualification is important: binary-valued tensors in ordinary PyTorch convolutions do not by themselves produce an XNOR speedup.

Recent diffusion quantization work has shown strong results at moderate bit widths [4], [5]. BiDM extends the question to

binary weights and activations, using a purpose-built structure and distillation from a full-precision teacher [6]. Its observation that naive full binarization is catastrophic motivates a narrower question: how much of that failure follows from the binary representation, and how much follows from projecting a floating-point solution into it after training?

We answer only at small scale. Rather than propose a new binary primitive, we perform a controlled comparison of three training paths: zero-shot PTQ, native training under binary forwards, and warm-started QAT. Parameter names, shapes, biases, boundary layers, optimizer, and sampling are held fixed. We also test two architectural explanations that appeared plausible in preliminary experiments: per-filter mean-centering and pre-activation residual ordering. Finally, we audit the evaluation itself. A classifier-confidence “legibility” score used in an earlier draft gives a 100% confidence floor on Gaussian noise and is therefore removed as a quality metric.

The study makes four contributions:

- a parameter-compatible comparison of FP32, native W1A32, PTQ W1A32, warm-started QAT, and a W1A1 core, each with three independent training seeds;
- a factorial MNIST ablation showing that pre-activation helps in the best native condition but mean-centering does not support the proposed geometric explanation;
- a calibrated distributional evaluation with real–real and noise controls, replacing an invalid confidence-based headline metric; and
- reproducible artifacts, including SHA-256 provenance and an actual packed binary-weight export whose whole-model payload is measured rather than inferred from the nominal bit width.

The result is a correction as much as a positive finding. Native W1A32 training is much more effective than zero-shot PTQ on MNIST, yet remains clearly behind the matched FP32 model. We therefore do not claim floating-point parity, a theorem about centering, or measured runtime/energy gains.

II. RELATED WORK

a) Diffusion.: A DDPM trains a time-conditioned network to predict noise added to a clean sample and samples through a Markov reverse process [1]. DDIM provides a non-Markovian alternative sampler [7]; improved objectives and

parameterizations can further raise quality [8]. We retain the original noise-prediction objective and ancestral sampler to minimize moving parts.

b) Quantized diffusion.: Q-Diffusion and PTQD address the unusual activation distributions and error accumulation of diffusion PTQ at 4–8 bits [4], [5]. BiDM studies binary weights and activations with a timestep-friendly architecture and spatially patched distillation [6]. Our tiny unconditional networks are not competitors to BiDM’s large-scale results. They instead isolate initialization and training path while holding the model topology constant.

c) Binary networks.: BinaryConnect and subsequent BNN work train latent real-valued parameters while using sign-projected weights during forward passes [2], [9]. XNOR-Net introduced magnitude scales that approximate real filters [3]. Gradients through discontinuous signs are commonly approximated by straight-through estimators (STEs) [10]. These established components form our binary layers; novelty is not claimed for the optimizer or quantizer.

III. CONTROLLED STUDY DESIGN

A. Objective and model

For clean image x_0 , Gaussian noise ϵ , and timestep t , all models minimize

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \left\| \epsilon - \epsilon_\theta \left(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, t \right) \right\|_2^2. \quad (1)$$

We use 1,000 linearly spaced diffusion steps. The denoiser is a compact residual U-Net with base width 16, widths [16, 32, 64], two downsampling stages, two skip concatenations, sinusoidal time embeddings, and GroupNorm. It has 218,993 parameters on MNIST. Input/output convolutions, normalization, time-conditioning linears, biases, scales, and residual additions remain FP32 in every condition. Consequently, “W1A1 core” is more accurate than “fully binary network.”

B. Binary forward pass

For output filter W_o , uncentered projection uses

$$\widehat{W}_o = \alpha_o \text{sign}(W_o), \quad \alpha_o = \text{mean}(|W_o|). \quad (2)$$

The centered variant replaces W_o by $W_o - \mu_o$, where $\mu_o = \text{mean}(W_o)$, before computing both sign and scale. Latent FP32 parameters are updated with an identity STE. In the W1A1 core, block activations additionally use a sign forward and clipped-identity backward pass. Boundary layers stay floating point to avoid discretizing pixels and predicted noise.

C. Conditions

Table I defines the interventions. “Native” means random initialization with binary weights used from the first forward pass. PTQ loads a converged FP32 state dictionary and changes only the forward projection, with no calibration or retraining. Warm QAT loads that same FP32 checkpoint and trains for the full binary budget. PTQ here is intentionally a zero-shot stress test, not a comparison against sophisticated diffusion PTQ methods.

TABLE I: Controlled conditions. Activations are FP32 unless marked binary.

Condition	Initialization	Core precision
FP32	random	W32/A32
Native W1A32	random	W1/A32
PTQ W1A32	FP32 final	W1/A32
Warm QAT W1A32	FP32 final	W1/A32
Native W1A1 core	random	W1/A1

For native W1A32 on MNIST we cross centered/uncentered projection with pre- and post-activation residual blocks. All variants have identical trainable parameter counts and compatible state dictionaries.

D. Data and optimization

We use the canonical 60,000/10,000 MNIST split [11] and the 50,000/10,000 CIFAR-10 split [12]. Images are scaled to $[-1, 1]$; CIFAR-10 training uses random crop and horizontal flip. Each condition is optimized in 12-epoch stages with AdamW, learning rate 2×10^{-4} , weight decay 10^{-4} , cosine decay, gradient norm clipping at 1, batch size 256 for MNIST and 128 for CIFAR-10, and seeds $\{0, 1, 2\}$. AMP accelerates training, while stored parameters and evaluation remain FP32. The sampler is ancestral DDPM with all 1,000 reverse steps. Each reported model distribution uses 2,000 samples from a fixed sampling seed; the training seed changes independently.

Native and FP32 models receive one optimization stage. PTQ adds no training after the FP32 stage, whereas warm QAT receives a second 12-epoch stage after loading FP32. Thus warm QAT uses 24 total training epochs; its initialization is not treated as free compute.

E. Evaluation and controls

An independently initialized domain classifier is trained only on the real training split. Its penultimate features define domain-feature Fréchet distance (dFID), unbiased cubic-polynomial kernel distance (dKID) [13], and $k = 5$ density and coverage [14]. We also report Jensen–Shannon divergence between mean predicted class distributions. Lower is better for dFID, dKID, and JS; higher is better for density and coverage. These are domain-feature metrics, not ImageNet Inception FID, and should not be compared numerically with standard FID leaderboards.

The MNIST encoder reaches 98.03% test accuracy and 1.44% ten-bin expected calibration error. A disjoint real–real split gives dFID 0.076, density 0.969, and coverage 0.971. Clamped Gaussian noise gives dFID 2278.24 and zero density/coverage. Yet the same noise receives mean peak classifier confidence 1.000, demonstrating why confidence is retained only as a diagnostic and not as a quality score.

The independently trained CIFAR-10 encoder reaches 91.51% test accuracy and 4.48% expected calibration error. Its real–real control gives dFID 7.13, dKID 0.002, density 1.013, and coverage 0.969; clamped Gaussian noise gives dFID 607.86, dKID 4.668, density 0.008, and coverage 0.001. These

controls make the poor absolute performance of the compact CIFAR-10 generators visible.

IV. MNIST RESULTS

A. Native training mitigates PTQ collapse, without FP parity

Table II gives the central result. The best native W1A32 condition reduces dFID from 1314.20 under matched uncentered PTQ to 46.33. Its nonzero density and coverage also show that the gain is not solely a shift in the first two feature moments. Nevertheless, FP32 is substantially better on every distributional metric. The defensible conclusion is therefore not “binary matches floating point,” but “the training path accounts for a large fraction of zero-shot one-bit PTQ failure on MNIST.”

Warm-started centered QAT improves upon centered native training in dFID (66.45 versus 99.50), so a floating-point initialization is not intrinsically harmful once binary forwards are optimized. Native training’s advantage is over zero-shot projection, not over every teacher- or warm-start-dependent method.

B. The centering hypothesis is rejected

Centering moves the mean positive-sign fraction closer to 0.5, but this balance does not imply better filter geometry or generation. Averaged over seeds, centered pre-activation weights have 34.05° latent-to-projected angular error versus 33.29° without centering. Corresponding dFID worsens from 46.33 to 99.50. Under post-activation the dFID means are similar and have large seed variance, while dKID is worse with centering. These observations directly contradict the earlier “structural dominance” account; we discard it rather than relabel the heuristic as theory.

Pre-activation is useful only conditionally. It improves the uncentered mean from dFID 84.26 to 46.33, but not the centered mean. This interaction and the large three-seed variance argue against presenting residual ordering as a general stability principle.

C. Activation binarization remains difficult

The W1A1 core is trained natively yet degrades to dFID 204.55 ± 69.77 . It remains better than zero-shot PTQ in dFID, but its very low density and coverage indicate a large distributional gap. Moreover, the model is not end-to-end binary because boundaries, normalization, time conditioning, scales, and accumulation remain floating point.

D. Storage accounting

Binary convolutions contain 195,328 of 218,993 parameters (89.19%). A physical export packs their signs with `numpy.packbits`, stores one FP32 scale per output channel, and retains all non-binary tensors as FP32. The resulting tensor payload is 121,316 bytes versus 875,972 bytes for an all-FP32 state, a $7.22\times$ whole-model reduction; the self-describing NPZ container is 143,350 bytes. This is smaller than the often-quoted $32\times$ figure because 10.81% of parameters, normalization state, and scales are not binary. It is a storage measurement, not a runtime or energy measurement.

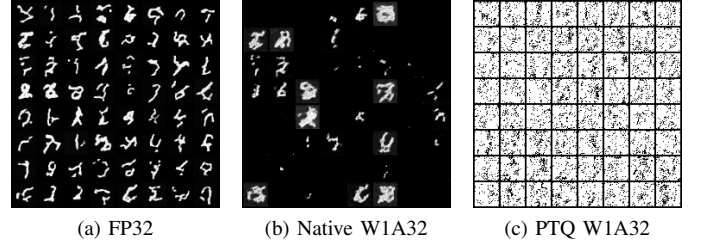


Fig. 1: MNIST samples from the fixed evaluation seed. The native binary model retains digit-like structure but is visibly below FP32; zero-shot PTQ collapses.

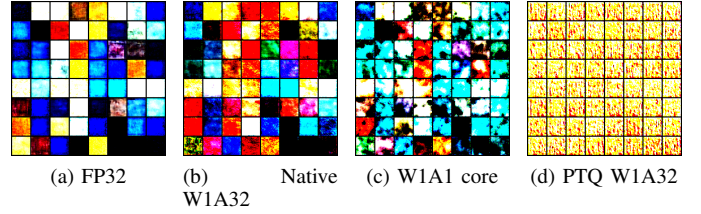


Fig. 2: CIFAR-10 samples from seed 0. Even FP32 produces coarse color blocks; the stress test is consequently not evidence of natural-image fidelity.

V. CIFAR-10 STRESS TEST

The same base-width-16 model and 12-epoch budget are applied to CIFAR-10 without architecture tuning. We evaluate FP32, centered and uncentered native W1A32, centered PTQ, centered warm QAT, and the W1A1 core over three seeds. This is a natural-image stress test of the MNIST observation, not a competitive CIFAR-10 generator.

Native optimization again avoids the worst zero-shot PTQ collapse. However, the FP32 model itself is hundreds of dFID points from the real-real control and has only 0.082 coverage. Although the W1A1 core has the best mean dFID, dKID, density, and coverage among these weak generators, its three-seed intervals overlap FP32 and its samples remain coarse (Fig. 2). The experiment therefore cannot adjudicate binary-versus-FP quality on natural images; it shows that this small architecture and budget are inadequate for that claim.

VI. DISCUSSION

a) *What the experiment supports.*: On MNIST, optimizing with the binary forward model is decisively better than zero-shot sign projection. This is consistent with the binary and floating-point objectives favoring different parameter regions. It does not establish that random initialization is uniquely necessary: warm-started QAT also recovers much of the PTQ loss. CIFAR-10 preserves the qualitative PTQ failure, but its failed FP32 control prevents a stronger precision comparison.

b) *What it falsifies.*: The experiments do not support FP parity, a centering theorem, a universally superior pre-activation topology, or the validity of peak classifier confidence as a

TABLE II: MNIST distribution metrics, mean $\pm$ sample standard deviation over three training seeds. All models use the same 2,000-sample ancestral-DDPM evaluation. Best non-FP result in bold.

Condition	dFID $\downarrow$	dKID $\downarrow$	Density $\uparrow$	Coverage $\uparrow$	Class JS $\downarrow$
FP32	3.15 $\pm$ 0.38	0.056 $\pm$ 0.013	0.284 $\pm$ 0.014	0.195 $\pm$ 0.017	0.033 $\pm$ 0.009
Native, uncentered, pre	46.33 $\pm$ 17.24	1.286 $\pm$ 0.816	0.022 $\pm$ 0.005	0.033 $\pm$ 0.010	0.295 $\pm$ 0.020
Native, centered, pre	99.50 $\pm$ 25.77	7.598 $\pm$ 3.223	0.010 $\pm$ 0.008	0.017 $\pm$ 0.014	0.308 $\pm$ 0.047
Native, uncentered, post	84.26 $\pm$ 61.39	6.881 $\pm$ 6.897	0.033 $\pm$ 0.051	0.028 $\pm$ 0.034	0.286 $\pm$ 0.113
Native, centered, post	82.46 $\pm$ 61.39	7.943 $\pm$ 8.043	0.037 $\pm$ 0.043	0.032 $\pm$ 0.025	0.253 $\pm$ 0.087
Warm QAT, centered, pre	66.45 $\pm$ 28.49	4.945 $\pm$ 3.686	0.019 $\pm$ 0.004	0.032 $\pm$ 0.004	0.320 $\pm$ 0.059
Native W1A1 core	204.55 $\pm$ 69.77	41.03 $\pm$ 25.01	0.005 $\pm$ 0.002	0.009 $\pm$ 0.004	0.339 $\pm$ 0.027
PTQ, centered, pre	871.12 $\pm$ 717.34	1581.31 $\pm$ 2433.16	0	0	0.533 $\pm$ 0.000
PTQ, uncentered, pre	1314.20 $\pm$ 314.96	2467.50 $\pm$ 1549.51	0	0	0.531 $\pm$ 0.002

TABLE III: CIFAR-10 stress test, mean $\pm$ sample standard deviation over three training seeds. The real-real control has dFID 7.13, dKID 0.002, density 1.013, and coverage 0.969. Bold marks the best model mean, not a claim of success.

Condition	dFID $\downarrow$	dKID $\downarrow$	Density $\uparrow$	Coverage $\uparrow$	Class JS $\downarrow$
FP32	382.67 $\pm$ 46.91	3.094 $\pm$ 0.353	0.230 $\pm$ 0.132	0.082 $\pm$ 0.046	0.146 $\pm$ 0.031
Native, uncentered, pre	402.46 $\pm$ 21.56	3.454 $\pm$ 0.347	0.125 $\pm$ 0.023	0.048 $\pm$ 0.005	0.143 $\pm$ 0.010
Native, centered, pre	384.32 $\pm$ 11.95	3.244 $\pm$ 0.136	0.183 $\pm$ 0.086	0.061 $\pm$ 0.011	0.168 $\pm$ 0.003
Warm QAT, centered, pre	403.61 $\pm$ 22.46	3.301 $\pm$ 0.194	0.155 $\pm$ 0.065	0.056 $\pm$ 0.022	0.180 $\pm$ 0.017
Native W1A1 core	365.02 $\pm$ 26.97	3.078 $\pm$ 0.237	0.293 $\pm$ 0.105	0.082 $\pm$ 0.014	0.152 $\pm$ 0.019
PTQ, centered, pre	564.21 $\pm$ 36.08	4.377 $\pm$ 0.282	0.059 $\pm$ 0.023	0.003 $\pm$ 0.001	0.262 $\pm$ 0.058

generative quality metric. Correcting those points narrows the claim but makes it testable.

c) *Relation to BiDM.*: BiDM addresses W1A1 diffusion at substantially greater resolution and uses specialized architecture and distillation [6]. Our results neither replace nor directly compare with that system. They show, in a matched tiny network, that zero-shot PTQ is an overly weak baseline for deciding whether binary weights can be trained. Scaling teacher-free binary diffusion remains unresolved.

VII. HARDWARE ARTIFACT: DIFFUINO

As a system demonstration separate from the controlled study, we deploy Diffuino on an Arduino UNO Q. The Qualcomm Linux processor executes a class-conditioned MNIST diffusion model with ONNX Runtime and streams selected states from the 1,000-step ancestral reverse process over Bridge RPC. The STM32 coprocessor refreshes the onboard 8×13 LED matrix, where each 28×28 state is rendered as a centered 8×8 grayscale digit. Four RGB LEDs report CPU load, memory load, denoising progress, and agreement between the requested class and a diagnostic classifier.

For a reliable interactive exhibit, the default artifact uses a separately trained class-conditioned FP32 checkpoint. Fixed per-class sampling seeds and display-only resize/contrast settings improve repeatable legibility on the small matrix; neither is used in the paper’s distributional evaluation. The study’s native W1A32 seed-2 checkpoint can also be exported to ONNX by materializing its sign-scale convolution weights as ordinary floating-point convolution tensors. Consequently, that path preserves the learned binary-weight representation but does not execute packed XNOR convolutions. Diffuino therefore demonstrates functional edge deployment and live

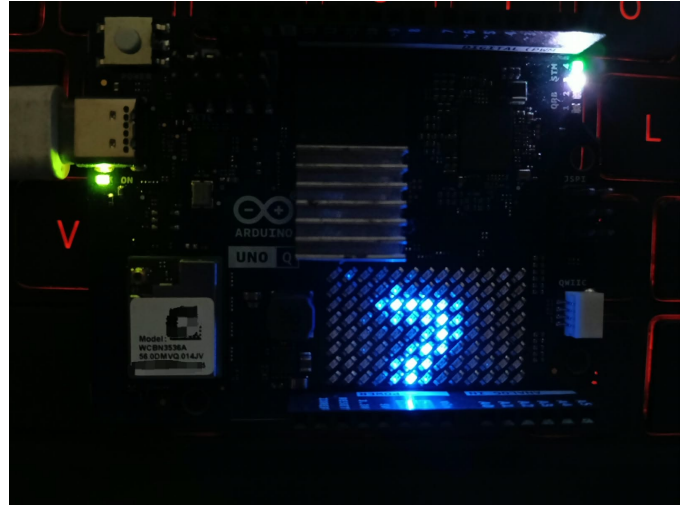


Fig. 3: Diffuino running on the Arduino UNO Q. The Qualcomm processor performs ONNX inference while the STM32 drives the 8×13 LED matrix and auxiliary RGB status LEDs. This artifact is separate from the controlled-study metrics.

diffusion visualization, not binary arithmetic acceleration, latency reduction, or energy savings. The installable app, packaging script, model files, and firmware are included with the released code.

VIII. LIMITATIONS

The network and training budget are intentionally small. Three seeds reveal large variance but are insufficient for fine-grained significance claims. Each metric uses 2,000 generated

samples, so absolute estimates are noisier than common 50,000-sample evaluations. Domain encoders make the metrics meaningful for digits, but their values are not comparable to standard ImageNet Inception FID. Classifier features can also miss perceptual defects, which is why sample grids and multiple density/diversity statistics are retained. The CIFAR-10 experiment is a failed small-model stress test rather than a tuned benchmark; apparent rankings within that low-quality regime should not be interpreted as natural-image progress. Finally, storage is physically packed, but no binary convolution kernel is implemented; latency, operation, and energy improvements remain unmeasured.

IX. REPRODUCIBILITY

The released pipeline records configuration, per-epoch loss, training and sampling seeds, package/CUDA/device versions, source commit, checkpoint SHA-256, parameter accounting, and quantization diagnostics beside every checkpoint. Atomic JSON writes prevent partial metadata. Evaluation artifacts include per-seed metrics, aggregate means and sample standard deviations, encoder accuracy/calibration, real-real and Gaussian-noise controls, and fixed-seed sample grids. Unit tests cover parameter compatibility, forward shapes, diffusion schedules, distribution-metric controls, and physical bit packing. The exact entry points are documented under `experiments/vl_binary_study`; all paper tables are traceable to `artifacts/vl_binary_study`. Diffuino is distributed as a deterministically generated Arduino App archive with an allowlisted runtime payload and a SHA-256 manifest; its interactive FP32 checkpoint is explicitly separated from the binary checkpoints evaluated in the study.

X. CONCLUSION

A binary-weight diffusion model should not be judged only by projecting a floating-point checkpoint to one bit. In our controlled MNIST study, training under binary forwards reduces domain-feature distance by more than an order of magnitude relative to zero-shot PTQ. The same experiment also places firm limits on that result: FP32 remains substantially better, binary activations cause a further gap, mean-centering does not help the strongest condition, and packed storage does not imply measured acceleration. CIFAR-10 further shows that the

present small-model recipe does not transfer into a credible natural-image generator. The useful conclusion is modest but reproducible: optimization path matters at one bit on MNIST, and future binary diffusion work should compare native, warm-started, and distilled training rather than treating zero-shot PTQ failure as the binary limit.

Code availability. Source, checkpoints, packed weights, and evaluation artifacts are available at <https://github.com/elixpo/monodiffuse>.

REFERENCES

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851.
- [2] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016.
- [3] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-Net: ImageNet classification using binary convolutional neural networks,” in *European Conference on Computer Vision*. Springer, 2016, pp. 525–542.
- [4] X. Li, Y. Liu, L. Lian, H. Yang, Z. Dong, D. Kang, S. Zhang, and K. Keutzer, “Q-diffusion: Quantizing diffusion models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 535–17 545.
- [5] Y. He, L. Liu, J. Liu, W. Wu, H. Zhou, and B. Zhuang, “PTQD: Accurate post-training quantization for diffusion models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [6] X. Zheng, X. Liu, Y. Bian, X. Ma, Y. Zhang, J. Wang, J. Guo, and H. Qin, “BiDM: Pushing the limit of quantization for diffusion models,” in *Advances in Neural Information Processing Systems*, 2024, arXiv:2412.05926.
- [7] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations*, 2021.
- [8] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2021, pp. 8162–8171.
- [9] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training deep neural networks with binary weights during propagations,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [10] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [12] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [13] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton, “Demystifying MMD GANs,” in *International Conference on Learning Representations*, 2018.
- [14] M. F. Naeem, S. J. Oh, Y. Uh, and Y. Choi, “Reliable fidelity and diversity metrics for generative models,” in *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020, pp. 7176–7185.